

Vibrations Workshop 10 - When the Wind Blows

By: Jake Peyser

ME301

4/29/2026

Overview

In this problem, a building with 4 floors is analyzed. Each floor is stacked on the one below. The horizontal dynamics of the floors (due to some external phenomena such as wind) can be modeled via linear springs attached to each floor relative to the one below. Each of the floors will have identical masses $m = 4000$ kg. The effective spring stiffness of each floor is likewise given by $k = 5000$ N/m. This situation has the following illustration:

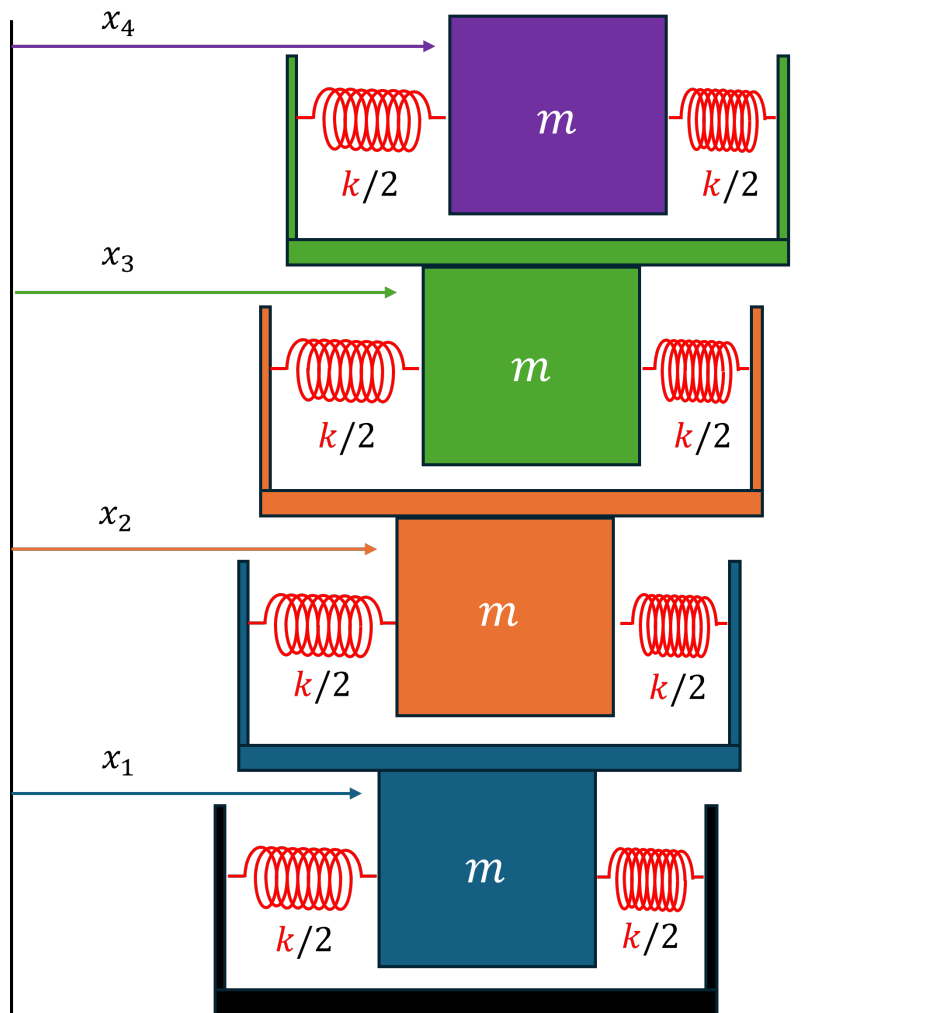


Figure 1: Physical Schematic

It will be assumed that each (simplified) floor behaves dynamically according to Newton's Second & Third Laws in conjunction with Hooke's Law. In this respect, there exists an equivalent formulation within the realm of circuits. In particular, with $v_i = \dot{x}_i$, in accordance with Kirchhoff's Voltage Law, the system has the following equivalent circuit:

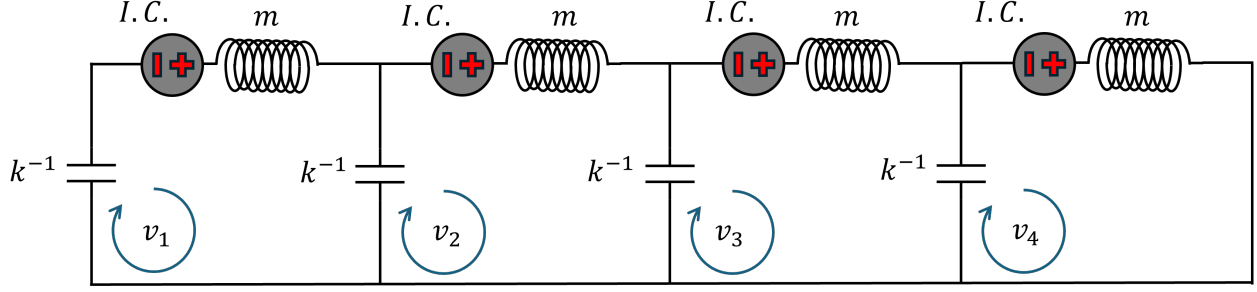


Figure 2: Equivalent Circuit

The *I.C.* force (voltage) generators correspond to the initial conditions needed to induce an initial offset of each of the masses relative to the fixed background.

If we artificially insert dampers with a corresponding damping constant R in parallel with each set of springs within each floor (note $R = 0$ as depicted presently in the above figure), then it follows directly from KVL in the laplace domain that the effective impedance matrix Z has the following representation:

$$Z(s) = \begin{bmatrix} ms + 2(k/s + R) & -k/s - R & 0 & 0 \\ -k/s - R & ms + 2(k/s + R) & -k/s - R & 0 \\ 0 & -k/s - R & ms + 2(k/s + R) & -k/s - R \\ 0 & 0 & -k/s - R & ms + k/s + R \end{bmatrix}$$

The advantage of inserting damping is that it helps visualize the resonate & anti-resonate frequencies of the system down the line in the frequency analysis. With some foresight, let $R = 1000$ kg/s.

Now, with the impedance matrix Z , the frequency response function $H : \omega \in \mathbb{R} \mapsto Z^{-1}(i\omega)/i\omega$ can be analyzed. In particular, using MATLAB (see Appendix), the component-wise modulus of the FRF as a function of frequency is as follows:

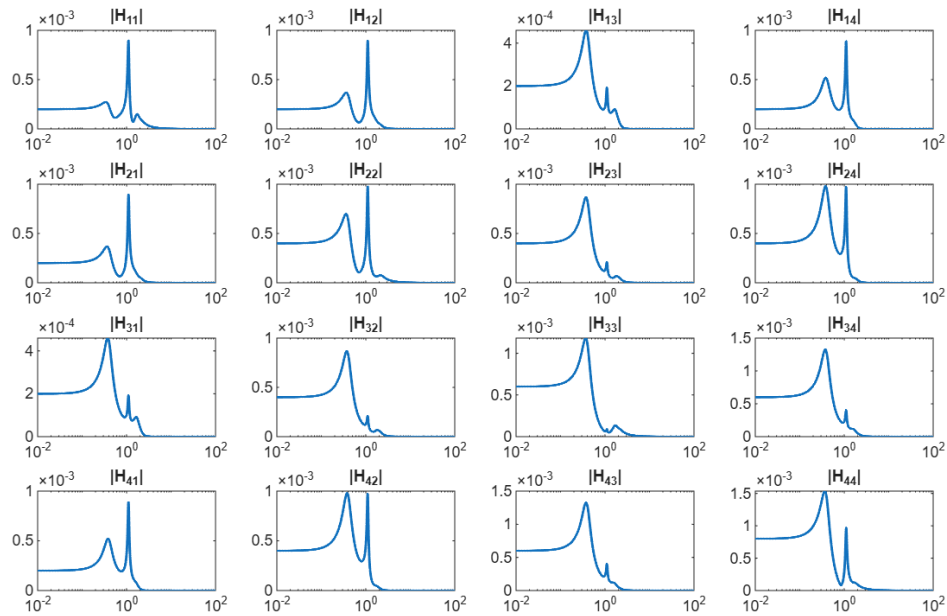


Figure 3: FRF Modulus

We likewise can plot the real and imaginary parts of the FRF. In that light, see the respective real and imaginary parts of H as a function of frequency below:

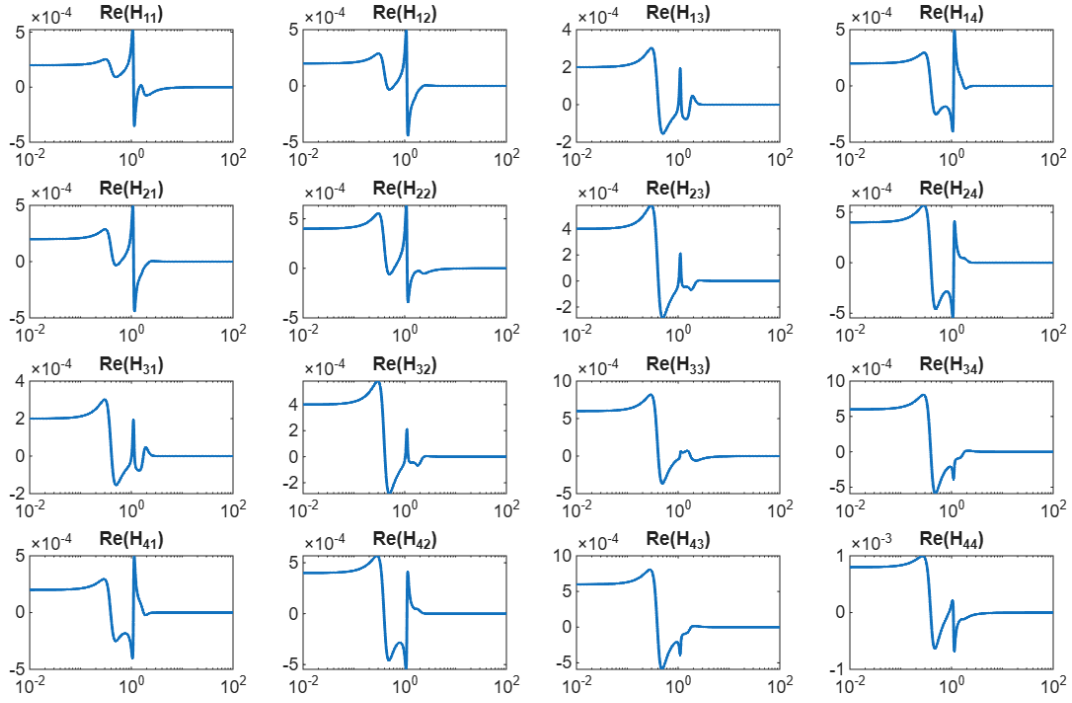


Figure 4: Real Part of FRF

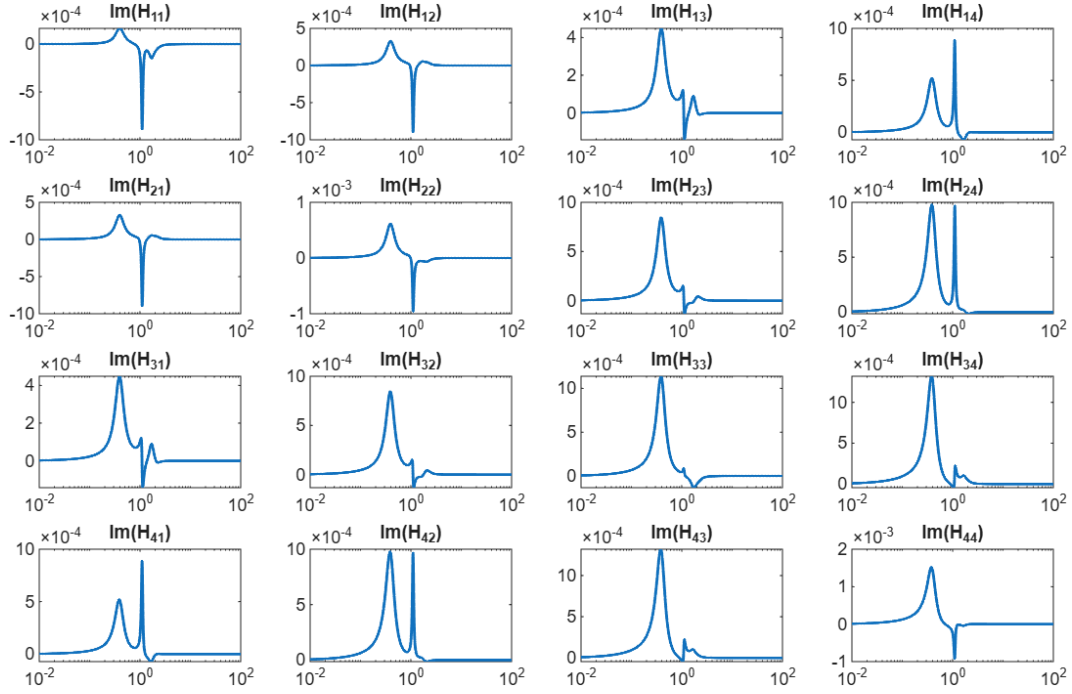


Figure 5: Imaginary Part of FRF

The model frequencies associated with the peaks/extrema of the imaginary part of the FRF were stored in a column vector inside MATLAB. The components of this vector, which correspond to those frequencies, were reported as such:

```

--- Modal Frequencies (rad/s) ---
0.3841
1.1040
1.6832
2.0531

```

Figure 6: Modal Frequencies

The corresponding amplitudes of each peak relative to the modal locations were likewise collected into a 4×4 matrix. The collection of the components of this matrix are reported as such:

```

--- Mode Shape Matrix A ---
0.3208    0.1580    0.1470    0.1210
0.6126    0.0382    0.1467    0.2049
0.8474    0.2547    0.0746    0.2217
1.0000    0.3217    0.2651    0.1581

```

Figure 7: Modal Amplitude Matrix

To parse what this matrix overlooks, the collection of (signed & normalized) amplitudes associated with each modal location (x_1, x_2, x_3, x_4 , and the ground) will be displayed in a mode shape diagram as illustrated below. Because the author of this document lacks the necessary skill to produce such a diagram, AI was used in its development (almost entirely).

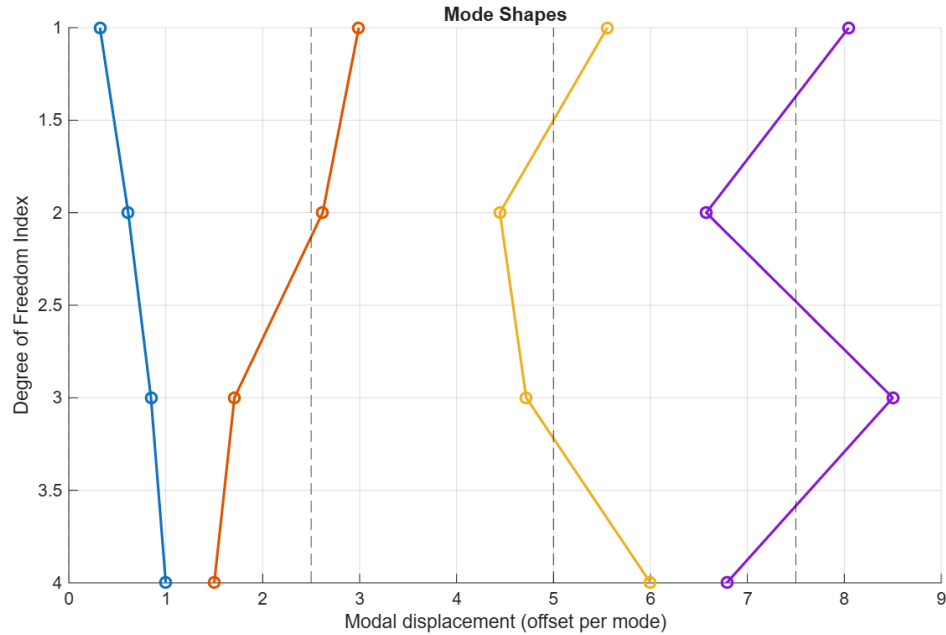


Figure 8: Mode Shapes as a *function* of Position

This mode shape diagram can be compared directly to the associated one in the text (Fig 4.10.). For reference, that figure is shown below:

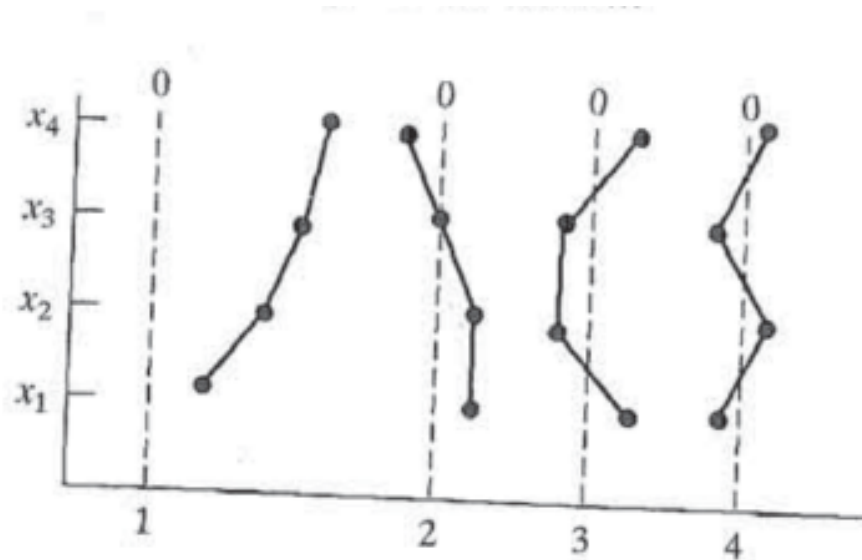


Figure 9: (Text: Fig 4.10.) Mode Shapes as a *function* of Position

Aside from what looks to be a sign error/discrepancy between the mode shapes presented here and in the one directly above, they do seem to agree somewhat with respect to, well, their shape. Of course the discrepancy between them is yet still a bit more deep rooted at positions 1 & 2. In particular, the amplitudes themselves appear smaller and out of order. No clue if this is due to the AI or if there is some other underlying error at play... Regardless, the agreement is pretty darn good at positions 2 & 3 visually.

Conclusionary Remarks

This problem established a methodology (modal analysis) of analyzing dominant frequencies in multiple degree of freedom systems. Using the FRF, these fundamental frequencies in conjunction with their peak amplitudes can be used to characterize the resonate dynamics of a system.

Appendix: MATLAB Code

```

1  clc; clear; close all;
2
3  %% Parameters
4  m = 4000;
5  k = 5000;
6  R = 1000;
7
8  %% Impedance Matrix
9  Z = @(s) ...
10     [m*s+2*(k/s+R),    -k/s-R,    0,    0;
11     -k/s-R,    m*s+2*(k/s+R),    -k/s-R,    0;
12     0,    -k/s-R,    m*s+2*(k/s+R),    -k/s-R;
13     0,    0,    -k/s-R,    m*s+k/s-R];
14

```

```

15 %% Control System Toolbox Implementation
16 s = tf('s');
17 Z_tf = Z(s);
18
19 %% External Force (Laplace)
20 F = [1 0 0 0];
21 x0 = [1 0 0 0];
22 v0 = [0 0 0 0];
23
24 %% Time Domain Parameters
25 t0 = 0;
26 t1 = 10;
27 dt = 0.001;
28 t = t0:dt:t1;
29
30 %% Coefficient Extraction
31 syms z
32 Z_sym = Z(z);
33
34 n = size(Z_sym,1);
35 M = zeros(n); K = zeros(n); Rmat = zeros(n);
36
37 for i = 1:n
38     for j = 1:n
39         Zij = Z_sym(i,j);
40         M(i,j) = double(limit(Zij/z, z, inf));
41         K(i,j) = double(limit(Zij*z, z, 0));
42         Rmat(i,j) = double(limit(Zij - M(i,j)*z - K(i,j)/z, z, 1));
43     end
44 end
45
46 %% Augmented Force Vector
47 F_aug = F' + M*v0' - (K*x0')/s;
48
49 %% System Response
50 Y = Z_tf \ eye(n); % FIXED (no inv)
51 V = Y*F_aug;
52 X = V/s + x0'/s;
53 H = Y/s; % FRF (displacement)
54
55 %% Time Simulation
56 u = zeros(size(t));
57 u(1) = 1/(t(2)-t(1)); % impulse
58
59 [v_resp, t_out] = lsim(V, u, t);
60 [x_resp, ~] = lsim(X, u, t);
61
62 v = squeeze(v_resp(:, :, 1));
63 x = squeeze(x_resp(:, :, 1));
64
65 %% Plot Velocities
66 figure; hold on; grid on;
67 for i = 1:n
68     plot(t_out, v(:, i), 'LineWidth', 1.4);

```

```

69 end
70 xlabel('Time (s)');
71 ylabel('Velocity (m/s)');
72 title('Velocity Responses');
73 legend(arrayfun(@(i) sprintf('v_%d',i),1:n,'UniformOutput',false));
74
75 %% Plot Displacements
76 figure; hold on; grid on;
77 for i = 1:n
78     plot(t_out, x(:,i), 'LineWidth',1.4);
79 end
80 xlabel('Time (s)');
81 ylabel('Displacement (m)');
82 title('Displacement Responses');
83 legend(arrayfun(@(i) sprintf('x_%d',i),1:n,'UniformOutput',false));
84
85 %% FRF via Nyquist Function
86
87 w = logspace(-2,2,2000);
88 [H_real, H_imag, wout] = nyquist(H, w);
89 H_mag = sqrt(H_real.^2+H_imag.^2);
90
91 % Real Parts
92 figure;
93 for i = 1:n
94     for j = 1:n
95         subplot(n,n,(i-1)*n + j); grid on;
96         semilogx(wout, squeeze(H_real(i,j,:)), 'LineWidth',1.4);
97         title(sprintf('Re(H_{%d%d})',i,j));
98     end
99 end
100
101 % Imag Parts
102 figure;
103 for i = 1:n
104     for j = 1:n
105         subplot(n,n,(i-1)*n + j); grid on;
106         semilogx(wout, squeeze(H_imag(i,j,:)), 'LineWidth',1.4);
107         title(sprintf('Im(H_{%d%d})',i,j));
108     end
109 end
110
111 % Modulus
112 figure;
113 for i = 1:n
114     for j = 1:n
115         subplot(n,n,(i-1)*n + j); grid on;
116         semilogx(wout, squeeze(H_mag(i,j,:)), 'LineWidth',1.4);
117         title(sprintf('|H_{%d%d}|',i,j));
118     end
119 end
120
121 %% Modal Analysis
122

```

```

123 Z = zeros(n);
124 I = eye(n);
125
126 % State-space system
127 A_ss = [Z, I;
128         -M\K, -M\Rmat];
129
130 % Eigen decomposition
131 [Psi, Lambda] = eig(A_ss);
132 lambda = diag(Lambda);
133
134 % Keep physical modes (positive imaginary part)
135 idx_phys = find(imag(lambda) > 0);
136
137 lambda_phys = lambda(idx_phys);
138
139 % Natural frequencies
140 omega = abs(imag(lambda_phys));
141
142 % Mode shapes
143 Phi = Psi(1:n, idx_phys);
144
145 % Normalize
146 for k_mode = 1:length(omega)
147     Phi(:,k_mode) = Phi(:,k_mode) / max(abs(Phi(:,k_mode))));
148 end
149
150 % Sort
151 [omega, idx_sort] = sort(omega);
152 Phi = Phi(:, idx_sort);
153
154 A = abs(real(Phi));
155
156 %% Display
157 fprintf('—— Modal Frequencies (rad/s) ——\n');
158 disp(omega);
159
160 fprintf('—— Mode Shape Matrix A ——\n');
161 disp(A);
162
163 %% Mode Shape Plot (AI generated)
164
165 Phi_plot = real(Phi);
166
167 % Normalize
168 for k_mode = 1:size(Phi_plot,2)
169     Phi_plot(:,k_mode) = Phi_plot(:,k_mode) / max(abs(Phi_plot(:,k_mode))));
170
171     % enforce consistent direction
172     if Phi_plot(1,k_mode) < 0
173         Phi_plot(:,k_mode) = -Phi_plot(:,k_mode);
174     end
175 end
176

```



```

177 % DOF positions (vertical axis)
178 y_pos = (1:n)';
179
180 figure; hold on; grid on;
181
182 spacing = 2.5; % horizontal separation between modes
183
184 for k_mode = 1:size(Phi_plot,2)
185
186     x_mode = Phi_plot(:,k_mode) + (k_mode-1)*spacing;
187
188     % dashed reference line (center of each mode)
189     xline((k_mode-1)*spacing, '—k');
190
191     % plot correctly ordered connections
192     plot(x_mode, y_pos, '-o', 'LineWidth', 1.5);
193 end
194
195 set(gca, 'YDir', 'reverse');
196
197 xlabel('Modal displacement (offset per mode)');
198 ylabel('Degree of Freedom Index');
199 title('Mode Shapes');

```